

The Interface Is a Variable: Measuring the Cost and Reliability of Purpose-Built UI Representations for LLM Agents

Working draft. Authors: TBD.

Status note (remove before submission): this is an in-progress draft. Sections backed by data are written; sections marked (*forthcoming*) await runs listed in [docs/paper-outline.md](#). Reported numbers are a single model (`gemini-2.5-flash`) on a single synthetic application and are explicitly provisional.

Abstract

Language-model agents increasingly operate software by consuming interfaces that were designed for people: screenshots, accessibility trees, and document object models. We ask whether the interface *representation* — how an application presents its state and available actions to an agent — is better understood as a control variable than as a fixed cost of automation. Holding the application, the task set, and an execution-based grader (a deterministic checker over backend state, never a model judge) constant, we compare four representations of the same store: a screenshot (C1), an accessibility tree (C2), a flat tool catalog (C3), and a purpose-built JSON *view document* (C4) that names the current view and the actions valid within it. We measure task success, token cost, step count, and illegal actions (actions the backend rejects), and report a cost–reliability frontier with paired per-task inference. On a first model, representations that reuse the human interface are clearly dominated: the screenshot (C1) costs roughly ten times the tokens of the cheapest representation and completes fewer than a third of tasks. Among the structured representations the trade-off is subtler and model-dependent: the view document (C4) reduces steps but, on this simple application and this model, does not yet justify its token overhead over a flat tool catalog (C3). We frame these results as a measurement method and a first data point, and identify where the advantage of a purpose-built agent surface is expected to emerge.

1. Introduction

An agent that operates software runs a loop: it observes the current state of an application, decides on one action, performs it, and observes again. Every observation the agent reads enters the model's context as input tokens, and every decision is emitted as output tokens. The application itself does not change across this loop; only the *representation* of it that the agent reads does.

Today that representation is almost always one built for humans. Agents are given screenshots to look at, or the accessibility and DOM trees that browsers expose for assistive technology, or a catalog of callable functions with no description of the current screen. Each of these inherits assumptions from its original purpose — pixels for human vision, verbose trees for screen readers, stateless function catalogs for programmatic callers — and none was designed to let an agent see, cheaply and unambiguously, *what is true now and what may be done next*.

This paper takes the position that the interface representation is a **control variable**, not an unavoidable cost of automation, and that a representation authored for the agent can change how many tokens a task costs and how reliably it completes. To test this cleanly we hold everything else fixed — one application, one task set, one deterministic grader, one model at a time, temperature zero — and vary only the representation across four conditions (Section 4). Two of these reuse the human interface (the screenshot (C1) and the accessibility tree (C2)); one is the common flat tool catalog (flat tools (C3)); and one is a purpose-built *view document* (C4) that states the current view, the entities in it, and the actions that are valid right now, with their argument constraints.

The applied motivation is the prospect that applications might one day expose an agent-facing surface alongside their human interface, much as they expose an API today. We treat that as motivation, not as our contribution. Our contributions are:

1. A controlled method for measuring interface representations for agents at **matched action grain**, so that differences are attributable to the representation rather than to a different set of operations (Section 4).
2. A **cost–reliability (Pareto)** framing — a representation is preferred only when nothing else is both cheaper and at least as reliable — with **paired per-task inference** (comparing conditions task-by-task with bootstrap confidence intervals), rather than a success-only leaderboard (Section 5).
3. First evidence on one application and model, including a **model-free observation-cost baseline** and an execution-based grader, showing that human-surface representations are dominated while the structured representations present a subtler, model-dependent trade-off (Section 6).

2. Important terms

This section defines the technical and coined terms used throughout the paper so that later sections can refer to them without re-deriving them. Each definition is consistent with how the term is used in the study.

- **Interface representation** — the form in which an application's state and available actions are presented to an agent; the study's independent variable.
- **Condition** — one of the four representations compared: screenshot (C1), accessibility tree (C2), flat tools (C3), and view document (C4).

- **View document (C4)** — a purpose-built JSON object stating the current view, its state and entities, and the affordances valid now, each with an **enabled** flag and an argument schema.
- **Flat tools (C3)** — a stateless catalog of callable functions with no current-view document; the model must track state itself.
- **Screenshot (C1)** — a rendered image of the human page; the agent acts via coordinates, typing, and scrolling.
- **Accessibility tree (C2)** — a text tree of the human page (roles and names); the agent acts on named elements.
- **Affordance** — an action the interface exposes, with an identifier, an **enabled** flag, and an argument schema.
- **Enabled flag** — a boolean on an affordance indicating whether it is valid in the current state.
- **Matched action grain** — the property that flat tools (C3) and the view document (C4) expose the same operations, so any difference between them is attributable only to the view document.
- **Illegal action** — an action the backend rejects (HTTP 400); a validity violation.
- **Malformed action** — a model response that is not a usable action (wrong shape); tracked separately from illegal actions.
- **Ignored-affordance rate** — how often a model attempts an action the interface marked unavailable (the t10 phenomenon).
- **Step / step cap** — one observe–decide–act cycle; the cap (20) bounds a run.
- **Execution-based grader** — a deterministic checker over backend state; never a model judge.
- **Cost–reliability (Pareto) frontier** — the trade-off of token cost versus success; a point is on the frontier if nothing is both cheaper and at least as reliable.
- **Paired per-task inference / bootstrap CI** — comparing conditions task-by-task with resampled 95% confidence intervals.
- **Observation cost** — the tokens an observation adds to context per step (text-tokenized for C2/C3/C4; image-tiled for C1).
- **Oracle control** — a deterministic perfect-agent baseline that validates the measurement pipeline.
- **Surface–backend consistency invariant** — the tested guarantee that the view document never advertises an action the backend would reject.
- **Refusal task** — a task whose correct behavior is to decline; reported separately from success tasks.

3. Related work

Reusing the human interface. A large body of work drives applications through representations built for people: agents that act on the DOM or accessibility tree, and screenshot- or pixel-based computer-use agents and models. These correspond to our screenshot (C1) and accessibility tree (C2) conditions. The relevant point for us is that they inherit a representation designed for human perception rather than for an agent's decision.

Optimizing the human representation. A separate line of work makes the human-derived representation cheaper for agents — for example pruning or serializing UI/accessibility trees to reduce their token footprint. This is adjacent but distinct: it *compresses a human-derived tree*, whereas we compare against a surface *authored from application state*.

Agent-facing protocols. Emerging efforts propose structured surfaces for agents. These run in different directions from ours: some describe an agent generating a user interface for a human to view; capability-catalog protocols expose a flat list of tools and readable resources — close to our flat tools (C3) — but do not define a per-view, stateful document with a-priori action validity, which is what our view document (C4) adds. None is a de-facto standard.

Benchmarks. Web and GUI agent benchmarks evaluate task success by *varying the agent or model while fixing the interface*. We do the opposite: we fix the task and grader and vary the interface. Our aim is not a leaderboard but an attribution of cost and reliability to representation.

(A verified reference list with URLs is maintained in [docs/research-plan.md](#), "Novelty and positioning"; it is reproduced in the References section below and will be formatted for the target venue.)

4. Study design

Application. MiniShop is a small store with a catalogue, product pages, a cart, and checkout. It is deliberately simple and its backend state is the single source of truth for grading.

Conditions. The same store is presented to the same model in four ways:

ID	Condition	Observation	Actions
C1	Screenshot	Rendered image of the human page	Click coordinates, type, scroll
C2	Accessibility / DOM	Text tree of the human page	Click/fill named elements
C3	Flat tools	Function catalog only; no current-view document	Call those functions
C4	View document	JSON: view, state, entities, affordances	Invoke an enabled affordance with typed arguments

The flat tools (C3) and view document (C4) conditions expose the **same operations at the same grain**. C3 gives the model a flat catalog of functions (`open_product` , `set_size` , `add_to_cart` , `go_checkout` , `set_address` , `pay` , ...) and, after each call, only a short status object; the model must track the application's state itself. C4 gives the same operations but adds, on every step, a document naming the current view, the relevant state and entities, and the affordances available now — each with an `enabled` flag and an argument JSON Schema. Because the action set is identical, any C3-vs-C4 difference is attributable to the view document.

Tasks and grader. Ten tasks (seven expected to succeed, three expected to be refused) are graded by a deterministic, execution-based checker over the backend order record — never by another model. Temperature is zero, the step cap is twenty, and the agent's history is limited to the task, its prior actions, and the current observation.

The C4 view document. The document is hand-authored as a function of backend state, which makes it an *upper bound* on how good an agent surface can be: it is faithful by construction. A consistency invariant, enforced by tests, guarantees that the document never advertises an action the backend would reject and that its size enumerations match stock exactly, so a favorable C4 result cannot be an artifact of a surface that disagrees with the application.

5. Metrics and analysis

Dependent variables. Per run: task success (binary; for refusal tasks, correctly declining); input and output tokens summed over steps (including image tokens on the screenshot (C1) where the provider itemizes them); step count; illegal actions (backend rejections); and a `malformed_actions` count (responses that are not a usable action), tracked separately so that model formatting failures are not silently absorbed.

Token accounting. Total input tokens decompose per step into a roughly constant overhead (system prompt, task, prior actions) plus the current observation, whose size varies by condition:

```

input_tokens(run) ≈ sum over steps of
    [ (system + task + prior actions)    # fixed overhead
      + current_observation ]            # varies by condition

```

Total cost is therefore approximately the per-step observation cost times the number of steps, which makes explicit that a representation can win either by being lighter per step or by needing fewer steps. Below we make precise how the observation term is counted for each condition, because the *text* conditions and the *spatial* (image) condition are counted in fundamentally different ways.

Text conditions (C2, C3, C4) — tokenizer-based. The observation is serialized text (an accessibility tree, a tool-result status object, or a JSON view document), so its token count is exactly the length of that serialized content under the model's tokenizer (we use `o200k_base` for the deterministic baseline). Cost therefore scales with *how much content the representation serializes*. The per-step decomposition is the same fixed overhead as above plus the condition-specific part: **flat tools (C3) additionally pays the full tool schema on every call** (the harness sends the function definitions with each request), and **the view document (C4) pays for that document** (view, state, entities, and per-affordance `enabled` flags and argument schemas). The accessibility tree (C2) pays for the verbose human tree. Reconstructing the canonical path deterministically, the median per-step input splits (`o200k_base`) are:

condition	system	task+prior	tool schema	observation	per-step total
C1 (screenshot, est.)	186	62	—	1105 (image)	1353
C2 (a11y tree, measured)	188	62	—	~650 (tree)	~900
C3 (flat tools)	159	62	490	25	736
C4 (view document)	188	62	—	422	672

The C3 row shows the tool schema (490 tokens) dwarfing its tiny status observation (25 tokens); the C4 row shows the document (422 tokens) as its whole condition-specific cost. C2's tree is *measured* from run traces because it requires a live browser to render; the other three are reconstructed deterministically with no model call.

Spatial/image condition (C1) — tiling-based, not tokenizer-based. A screenshot is not tokenized as text. Vision models charge image tokens by a **tiling** model that is a function of the image's *resolution*, not its content. For the OpenAI gpt-4o family the rule is: downscale the image to fit within a 2048×2048 box, then scale its shortest side to ~768px, then cover the result in 512×512 tiles, and charge

```

image_tokens ≈ base + per_tile × tiles

```

with high-detail `base = 85` and `per_tile = 170` (low detail is a flat 85 regardless of resolution). Two implications are central to this study:

1. **Image-token cost is a function of resolution/viewport, not of visual complexity or information content.** A blank page and a busy page rendered at the same viewport produce the identical tile count and therefore the identical cost. A screenshot cannot get *cheaper* by showing less, only by being sent at a lower resolution or in low detail — the opposite of the text conditions, where a leaner representation is directly cheaper. Because of the 768px shortest-side rescale, high-detail cost also **plateaus**: at our 1280×800 viewport it is 6 tiles (≈1105 tokens), and wider 16:9 viewports (1536×864, 1920×1080) still cost 6 tiles.
2. **Provider reporting differs, so C1's cost must be read from total input tokens.** OpenAI *itemizes* image tokens in a dedicated usage field, whereas Vertex/Gemini *folds* them into `prompt_tokens`. This is why our observed `gemini-2.5-flash` C1 runs report `image_tokens = 0` yet an input of ~46k tokens per task: the image is fully billed, just not itemized. The harness reads a dedicated `image_tokens` field when present and otherwise leaves it null (`harness/openai_compat.py::extract_usage`); comparisons anchor on total input tokens and treat the itemized image count as secondary.

These tiling constants are **provider- and model-specific** — Anthropic and Google use different resolution rules and per-tile costs — so we present the gpt-4o numbers as a concrete, labeled estimate and do not claim a single universal image-token formula. `harness/obs_cost.py` computes the estimate across a resolution sweep and the per-condition composition above deterministically; `harness/make_figures.py` renders them as `image_tokens_vs_resolution.png` (the resolution curve) and `token_composition_by_condition.png` (the stacked composition).

Analysis. We report a per- (`model`, `condition`) table and a cost–reliability Pareto frontier (median input tokens vs success rate). Because the same tasks run under every condition, we compute **paired per-task** differences between C4 and C3 with bootstrap 95% confidence intervals resampled over tasks, reporting success and refusal tasks separately.

Latency. (*forthcoming*) Latency is a planned metric and is not yet instrumented. It is expected to weight *round trips* (step count) differently from token cost, and could therefore move the C3-vs-C4 comparison; we will add per-call timing and report median time-to-completion per condition.

Procedures

This subsection states the exact, repeatable procedures behind the numbers. It does not re-derive the study design (Section 4) or the token accounting above; it records *how a run executes*, *how each metric is computed*, *how the data is analyzed*, and *how to reproduce all of it*.

Experimental procedure (one run). A "run" is one (condition, task, repeat) triple. The harness holds everything in Section 4 fixed — the MiniShop backend and its execution grader, the ten-task set (seven success, three refusal), temperature 0, a step cap of 20, and a history limited to the task text, the prior actions, and the current observation — and varies only the representation across the four conditions screenshot (C1), accessibility tree (C2), flat tools (C3), and view document (C4). Within a run the agent loops: observe the current representation, emit one action, apply it against the backend, and observe again, until the task completes or the step cap is reached. A full sweep is the Cartesian product 4 conditions $\times$ 10 tasks $\times$ R repeats (here R = 5, so 200 runs); each condition is launched as its own repeats invocation so a crash in one condition cannot abort the others. A deterministic **oracle** agent that replays the correct policy is run first as a control and end-to-end pipeline check (it reaches 100% on C3 and C4), separating apparatus correctness from model behavior. Every run is written to its own JSONL trace file (a run header, one row per step with the action and provider usage, and a final result row with the per-run rollups), so runs are independent artifacts that the analysis layer reads after the fact.

Measurement procedure (per metric). *Success* is the deterministic grader's verdict over the backend order record — for success tasks the order must match the task specification; for refusal tasks no forbidden order may be placed — never another model. *Input and output tokens* are the provider's reported usage summed over the run's steps; for the screenshot (C1), image tokens are read from total input tokens because Vertex folds them into `prompt_tokens` (Section 5). The per-condition observation cost is counted two different ways, as Section 5 specifies: the text conditions (C2, C3, C4) are counted by tokenizer on the serialized observation, while the screenshot (C1) is counted by the resolution-driven image-tiling model. *Steps* is the number of model calls until success or the cap. *Illegal actions* are backend rejections; *malformed actions* are non-object model responses (a JSON array or prose) that are not a usable action. The two error signals are counted separately — a malformed response is recorded as such rather than silently coerced to a no-op — so that a model's formatting failures are never absorbed into its illegal-action rate.

Analysis procedure. The analysis layer reads the final result row of every trace and collapses runs into per- (`model, condition`) cells, never pooling across models. For each cell it reports the success rate, median steps, median input/output/image tokens, and the illegal- and malformed-action rates per step. Uncertainty is a percentile bootstrap: the success rate and the median input tokens carry 95% CIs resampled (fixed seed) over the cell's task-repeat units. The cost–reliability frontier marks each cell as on- or off-frontier (a cell is dominated if another is no more costly and at least as reliable). Because the same tasks run under every condition, the C4-vs-C3 comparison is **paired per task**: average each task's outcome over its repeats under each condition, take the C4–C3 difference, and bootstrap a 95% CI over tasks — reported separately for success and refusal tasks so a refusal passed by inaction cannot inflate the headline.

Reproducibility checklist. Every number above is regenerated from the repository with no hidden state, in this order:

1. `python -m harness.scripted` — the non-model sanity check; a scripted policy must complete every success task on C3 and C4 and be refused on the refusal tasks before any model is run.
2. `python -m harness.obs_cost` — the model-free observation-cost baseline and the per-condition token composition (Section 6.1), `o200k_base` tokenizer, no API call.
3. `python -m harness.model_loop --oracle --conditions C3,C4` — the deterministic pipeline control (Section 6.2).
4. `python -m harness.model_loop --repeats 5` (per condition) — the model sweep that writes the 200 per-run traces (Section 6.3).
5. `python -m harness.report --figures` — aggregates the traces into the tables, bootstrap CIs, the paired comparison, and the Pareto/paired figures.
6. `python paper/build.py` — renders this manuscript (`paper.md` → `paper.html` / `paper.pdf`).

The environment (Python dependencies, the Playwright Chromium used for the C1/C2 browser render and for `paper/build.py`, and the model registry in `harness/models.py`) is pinned under `.cursor/`, so the sequence above runs identically on a fresh checkout.

6. Results

We report the results in three layers, from the model-free lower bound to the model run itself. Section 6.1 is a deterministic observation-cost baseline that needs no model. Section 6.2 is the oracle control that validates the pipeline. Section 6.3 is the first model run — `gemini-2.5-flash`, $N=5$ — read one metric at a time (success, token cost, steps, illegal/malformed actions), then broken out per task, then analyzed as a paired C4-vs-C3 comparison, and finally summarized as a per-condition error analysis. Everything here is one model on one synthetic application with ten tasks, so we read the coarse orderings and treat the fine C3-vs-C4 margin as provisional.

6.1 A model-free observation-cost baseline

Before any model is run, we measure deterministically how many tokens each condition's observation costs per step, replaying each task's canonical path and counting tokens with a fixed tokenizer. Median per-step observation cost is approximately 1105 tokens for the screenshot (C1) estimate, 512 for flat tools (C3) (which re-sends the tool schema each step), and 453 for the view document (C4). Text representations are thus about $2.4\times$ cheaper per look than a screenshot, and the view document is slightly cheaper per step than the flat tool catalog. This is the observation term only, along the minimal path; it does not include step-count effects, which the model run adds.

6.2 Pipeline control (oracle)

A deterministic "oracle" agent that replays the correct policy validates the full measurement pipeline end to end and provides a best-case reference: both flat tools (C3) and the view document (C4) reach 100% success. This isolates apparatus correctness from model behavior, so any success deficit in the model run below is attributable to the model, not to the harness or grader.

6.3 First model run: `gemini-2.5-flash` (N = 5)

We run one general model, `gemini-2.5-flash` (via Vertex, temperature 0, step cap 20), across all four conditions on all ten tasks, five times each — 200 runs in total. Table 1 reports the per-condition aggregate; the subsections that follow read it one metric at a time.

Table 1. Per-condition results, `gemini-2.5-flash`, N=5 (95% CIs bootstrapped over the 50 task-repeat units per cell).

Condition	Success (95% CI)	Median input tokens (95% CI)	Median steps	Illegal/step	Malformed/step
screenshot (C1)	30% [18, 42]	~46,338 [46,324, 46,368]	20	0.000	0.000
accessibility tree (C2)	72% [60, 84]	~6,770 [5,835, 9,714]	7	0.198	0.116
flat tools (C3)	100% [100, 100]	~3,134 [3,118, 3,154]	7	0.000	0.000
view document (C4)	92% [84, 98]	~4,455 [4,413, 4,491]	6	0.016	0.000

Success

The two human-surface conditions are clearly dominated on success. The screenshot (C1) passes only 30% of runs [18, 42] and none of the seven success tasks (its passes are all refusal tasks). The accessibility tree (C2) reaches 72% [60, 84] but is the least trustworthy number in the table, because its failures mix backend rejections and malformed responses (below). The two structured conditions clear the human surface comfortably: flat tools (C3) is perfect (100% [100, 100]) and the view document (C4) is near-perfect (92% [84, 98]), its only deficit isolated to task t10 (analyzed below). Read cautiously: with five repeats over ten tasks these intervals are wide, and the C3-over-C4 success gap is a single task.

Token cost

Input-token cost separates the conditions by an order of magnitude, and the mechanism is the per-step observation decomposition of Section 5 (per-step observation cost $\times$ steps + fixed overhead) — we do not repeat that accounting here. The screenshot (C1) is by far the most expensive at a median $\sim 46,338$ input tokens per task, roughly $10\times$ the view document (C4) and $\sim 15\times$ flat tools (C3); because Vertex folds image tokens into `prompt_tokens`, that cost is read from total input tokens, not from the itemized `image_tokens` field (which reads 0). Among the text conditions, flat tools (C3) is cheapest ($\sim 3,134$), the view document (C4) is next ($\sim 4,455$), and the accessibility tree (C2) is most expensive and most variable ($\sim 6,770$ [5,835, 9,714]) because tree size tracks page content. Notably, the model-run ordering of C3 below C4 *reverses* the minimal-path baseline of Section 6.1 (where C4 was cheaper per step): once real step counts are in play, the C4 document grows as the cart fills, so its mid-session documents outweigh C3's compact status objects.

Steps

Step count is where the view document (C4) helps. C4 uses the fewest median steps (6) and flat tools (C3) one more (7); paired per-task analysis (below) confirms the -1.17 -step difference is significant. The screenshot (C1) sits at the step cap (median 20) because it never completes a success task and simply exhausts the budget. The accessibility tree (C2) has a median of 7 but a long tail: it too hits the cap on the multi-item task t09.

Illegal and malformed actions

We track two distinct error signals: illegal actions (the backend rejects an attempted action) and malformed actions (the model emits a non-object response — a JSON array or prose — that is not a usable action). The screenshot (C1) and flat tools (C3) are clean on both (0.000 each). The view document (C4) has a small illegal rate (0.016/step), concentrated entirely on t10. The accessibility tree (C2) is the outlier on both axes — 0.198 illegal/step and 0.116 malformed/step — so its 72% success masks a genuinely noisy interaction: the human tree invites named clicks and fills on elements the backend then rejects, and the model periodically returns responses the loop cannot parse. Keeping the malformed count separate (rather than silently coercing it to a no-op) is what makes C2's error composition honest.

Per-task breakdown

Table 2 breaks the run down per task so the patterns behind the aggregates are visible. Each cell packs the three per-task medians for that condition: **success rate** $\cdot$ **median steps** $\cdot$ **median input tokens**, computed over the five repeats (refusal tasks are marked *r*).

Table 2. Per-task readout, `gemini-2.5-flash`, $N=5$. Cell = success rate $\cdot$ median steps $\cdot$ median input tokens.

Task		screenshot (C1)	accessibility tree (C2)	flat tools (C3)	view document (C4)
t01		0% · 20 · 46,368	20% · 12 · 11,702	100% · 7 · 3,154	100% · 6 · 4,498
t02		0% · 20 · 46,368	100% · 11 · 10,809	100% · 7 · 3,154	100% · 6 · 4,486
t03	<i>r</i>	100% · 2 · 4,184	100% · 2 · 1,997	100% · 2 · 1,081	100% · 2 · 1,597
t04		0% · 20 · 46,268	0% · 6 · 5,835	100% · 7 · 3,118	100% · 6 · 4,423
t05	<i>r</i>	100% · 20 · 46,018	100% · 3 · 2,605	100% · 2 · 713	100% · 2 · 1,528
t06		0% · 20 · 46,328	100% · 10 · 9,714	100% · 7 · 3,144	100% · 6 · 4,491
t07	<i>r</i>	100% · 20 · 46,348	100% · 2 · 1,990	100% · 2 · 1,079	100% · 1 · 905
t08		0% · 20 · 46,324	100% · 7 · 6,690	100% · 7 · 3,124	100% · 6 · 4,413
t09		0% · 20 · 46,428	0% · 20 · 21,658	100% · 12 · 5,833	100% · 10 · 7,867
t10		0% · 20 · 46,368	100% · 7 · 6,770	100% · 7 · 3,154	20% · 6 · 4,544

Four patterns stand out. **The screenshot (C1) hits the step cap (20) on every task it does not refuse** and fails all seven success tasks; its only passes are the refusal tasks (t03 by an early refusal, t05 and t07 by exhausting the budget without placing the forbidden order), and its cost is a nearly flat ~46k tokens because image cost is set by viewport, not content. **The accessibility tree (C2) is erratic**: it fails t01 (20%), t04 (0%), and the two-item t09 (0%, step-capped at 20 with the heaviest token load of any text-condition cell, ~21.7k). **Flat tools (C3) is uniformly clean** — 100% on every task at ~3k tokens and ~7 steps. **The view document (C4)'s deficit is isolated to t10 (20%)**; on every other task it matches C3's success while using one fewer step (6 vs 7, and 10 vs 12 on t09) and consistently more input tokens.

Paired C4-vs-C3 analysis

Because the same tasks run under every condition, we compare the view document (C4) against flat tools (C3) paired per task: for each task we average the outcome over its five repeats, take the C4–C3 difference, and bootstrap a 95% CI over tasks (fixed seed), reporting the seven success tasks and three refusal tasks separately. Table 3 gives all three metrics.

Table 3. Paired C4–C3 per-task differences, `gemini-2.5-flash`. Positive = C4 is higher.

Task set	n	Metric	Mean diff (C4 – C3)	95% CI	Excludes 0?
success	7	success	-0.114	[-0.343, 0.000]	no
success	7	input tokens	+1,404	[+1,317, +1,549]	yes
success	7	steps	-1.17	[-1.51, -1.00]	yes
refusal	3	success	0.000	[0.000, 0.000]	no
refusal	3	input tokens	+386	[-174, +815]	no
refusal	3	steps	-0.33	[-1.00, 0.00]	no

On the success tasks the view document **significantly reduces steps** (-1.17 per task; CI excludes zero — the whole point of a current-view/affordance document) but **significantly increases input tokens** (+1,404 per task; CI excludes zero, and every one of the seven success tasks is positive) and does **not** significantly change success (the CI [-0.343, 0.000] includes zero; the entire point estimate is the t10 deficit). On the axes we measured, flat tools (C3) is therefore Pareto-preferred here — cheaper and at least as reliable — so H2's specific claim that C4 dominates is not supported for this model and application. On the refusal tasks nothing separates the two conditions: both refuse correctly on all three, and the token and step differences are small with CIs that include zero. C4 keeps the fewest steps throughout, but on this small application that step saving does not offset its per-step token cost.

What the t10 failure means, plainly

Task t10 asks the agent to buy an item and ship it to an address. The valid order of operations is: open the product, add it to the cart, go to checkout, and *then* enter the shipping address — an address cannot be entered before checkout is reached. On this task the model tried to enter the address too early, while still on the product page. Because setting an address is only valid at checkout, the view document (C4) had explicitly labeled that action unavailable (`enabled: false`) at that step, and the backend refused it. The key point is what kind of failure this is: it is not a bug in the interface — the document was correct and agreed with the backend (an invariant we enforce with a test) — it is the model choosing to ignore a rule the interface clearly stated. This is central to the idea of the view document (C4): its value is that it tells the agent what is allowed right now, so a model that reads and respects those labels avoids the mistake, while a model that ignores them (as this one did) gains nothing from the extra information. It also gives a concrete quantity to measure across models: how often a model attempts an action the interface marked unavailable — an "ignored-affordance" rate. The failure is reproducible: C4 fails t10 on four of five repeats in the main sweep (all via the identical early `set_address`), and a diagnostic t10-only re-run reproduced it on five of five.

6.4 Second model (RQ3) — (*forthcoming*)

Whether "flat tools (C3) is Pareto-preferred" generalizes beyond one model is open. A second general model (e.g. a mid-tier Claude or Grok) will test in particular whether other models obey the `enabled` flag on the t10 path — that is, whether the ignored-affordance rate above is a property of this model or of the task.

6.5 Constraint-pruning ablation — (*forthcoming*)

To explain the view document (C4)'s token overhead and separate *compactness* from *constraint-carrying*, we will strip the `enabled` flags and argument enumerations from the C4 document (retaining view and entities) and measure whether the step savings survive without the constraint information.

7. Discussion

The provisional headline — that on the simplest application, with a model that ignores constraints, a flat tool catalog (C3) is preferred to the view document (C4) on tokens and success — should be read carefully. It is the *least favorable* setting for a purpose-built surface: the application's state is tiny and the tasks are short, so the burden C3 places on the model (tracking state itself) is light, while C4 pays a per-step cost to send a document the model barely needs.

We therefore expect a crossover as conditions become less favorable to C3: with more complex applications (more views, more state, more opportunities for invalid actions), with models that respect the document's constraints, when cost is weighted by latency (where C4's fewer round trips help rather than hurt), and in safety-sensitive settings where preventing illegal actions is itself valuable. C4 already shows the mechanism here: it uses fewer steps and keeps a near-zero illegal-action rate.

The t10 result (§6.3, "What the t10 failure means, plainly") is a finding in its own right: a truthful `enabled: false` flag was present and ignored. That the document *told the truth* — and agreed with the backend, an invariant we test — and the model *still acted against it* is precisely the kind of agent-experience signal a metric suite should capture. It also names a concrete, model-comparable quantity, the **ignored-affordance rate** (how often a model attempts an action the interface marked unavailable), which the second model run (§6.4) is designed to measure and which motivates the second paper.

8. Adopting the view document (C4): paths and complexity for existing and future applications

The study measures a hand-authored view document (C4) as an upper bound, which naturally raises the practical question of what it would take for a real application to expose such a surface. This section addresses that question directly. It is not an evaluation — the numbers in Section 6 speak only to the hand-authored case — but a structured account of the adoption paths available and their relative complexity, so that a reader can judge where the trade-off in Section 7 (a per-step token cost now versus a perpetual runtime state-tracking tax) is likely to be worth paying.

Core principle: a faithful projection, not new business logic. The view document (C4) is a *faithful projection of backend state*. The backend remains the single source of truth, and the document is a re-serialization of what the backend already knows: which view the agent is on, the entities in it, and the actions that are valid right now with their argument constraints. The surface-vs-backend consistency invariant we enforce with tests (Section 4) is the formal statement of this: the document may never advertise an action the backend would reject. The consequence for adoption is the central, and encouraging, observation of this section: **almost all of the information a C4 document carries is validity logic the application already has**. A human interface already disables the "Add to cart" button until a size is chosen, already validates the address field, already hides the "Pay" button until the cart is non-empty and the address is set. Those enable-conditions, form validations, and in-stock checks are the same predicates a C4 document serializes into its `enabled` flags and argument schemas. Adoption is therefore largely a *serialization* problem — expose logic that already exists in a machine-readable shape — rather than a request to author new business rules. This reframing is what makes the paths below tractable.

8.1 A spectrum of adoption paths

Applications can produce a C4 surface in several ways, which differ sharply in who pays the authoring cost, how much of the application they cover, and how faithful the result is. We describe five, from the highest-leverage (available to applications built with agent surfaces in mind) to the most general (available to any application, at a cost). Each carries an explicit complexity rating on three axes: *effort* (what a developer must do), *coverage* (how much of a real application it reaches), and *fidelity* (how faithfully the surface tracks backend truth).

1. Framework-emitted (future applications). A UI framework in which the developer declares views, state, and actions once, and the framework emits *both* the human render and the view document (C4) from that single declaration. Because the enable-conditions and validation are already written for the human UI, the framework can project them into the agent document with no additional developer work per view. *Effort: low at the margin (a one-time framework investment, then near-zero per application). Coverage: high. Fidelity: high (the two surfaces are generated from one declaration, so they cannot drift)*. This is the highest-leverage path but presupposes framework support that does not yet exist widely.

2. Derived from a structured backend. Many applications already hold their capabilities in machine-readable specifications: a GraphQL schema, form and validation schemas, or a server-driven UI description. Where such a spec exists, C4 can be *projected* from it — the types become entities, the mutations become affordances, the validation rules become argument constraints and `enabled` flags. *Effort: low–medium (write and maintain the projection; reuse existing specs).* *Coverage: medium–high, bounded by how much of the app is actually spec-driven. Fidelity: high where the spec is authoritative, lower where the UI adds validity logic the spec does not capture.*

3. Hand-authored per view (what this study does). A developer writes the view document by hand for each view, as a function of backend state. This is faithful by construction and is exactly the arm we measure, but it does not scale: every view is bespoke work, so it suits a small number of high-value applications or flows rather than a whole ecosystem. The pilot gives a concrete data point for the per-view cost. In MiniShop, the C4 projection (`minishop/surface.py`, `build_surface`, ≈ 140 lines) is roughly 3–4 $\times$ the size of the flat-tools (C3) definition (`minishop/tools.py`, ≈ 40 lines). That multiple is the honest cost of the document — but it is *mirroring* logic the human templates already encode (the same size/stock and view checks that gate the human buttons), not inventing new rules, which is why even the hand-authored path is closer to transcription than to design. *Effort: medium per application. Coverage: whatever is authored. Fidelity: high (authored directly against backend state and pinned by the consistency test).*

4. Compiled from the human surface (existing and legacy applications). For an application that already ships a human UI and cannot be rebuilt, a compiler can read the rendered surface — the DOM, ARIA roles, and `disabled` attributes — and emit a view document from it. The attractive property is that it costs the application *nothing*: no new endpoint, no re-declaration, no backend change. The cost is fidelity. A compiled surface inherits whatever the human page happens to expose, so it can be *lossy* (state the DOM does not render is invisible to it) and it can *disagree* with the backend (a button enabled in the DOM that the backend would nonetheless reject). A compiled document must therefore be validated against the backend before it is trusted, exactly as our consistency invariant validates the hand-authored one. *Effort: near-zero for the application (the work is in the compiler). Coverage: broad — it applies to any app with a human UI. Fidelity: lossy, and must be checked.*

5. Model-extracted (a model reads the screen). The most general path: an LLM observes the human surface (a screenshot or the accessibility tree) and *emits* the view document itself. It applies to any application with no cooperation whatsoever, which is its whole appeal. But it inverts the economics of C4. The document's purpose is to pay *once* so the agent does not re-derive state at runtime; extracting it with a model moves that cost back to runtime and pays it on every step, and it reintroduces exactly the hallucination risk — an advertised action the backend rejects — that a faithful projection removes. It is best understood as a fallback for surfaces reachable no other way. *Effort: near-zero for the application. Coverage: universal. Fidelity: lowest — subject to model error, and cost recurs at runtime.*

8.2 Cost framing: build-time versus runtime

The five paths above trade a single quantity against another. The view document (C4) pays its cost *once*, at build or framework time (paths 1–3) or at compile time (path 4); thereafter every agent interaction reads a surface that already states what is true and what is allowed. Flat tools (C3), by contrast, pays a *perpetual runtime tax*: because it ships no current-view document, the model must re-derive the application's state from its own history on every step, for the life of the deployment. Adoption effort is thus a one-time (or amortized) investment weighed against a recurring per-interaction cost. Two things move that balance toward adoption. First, scale: a framework- or spec-derived surface amortizes its cost across every application and every run, so the per-interaction saving compounds. Second, complexity: the runtime state-tracking tax that C3 imposes grows with the number of views, the amount of state, and the number of ways an action can be invalid — precisely the regime (Section 7) where we expect C4's advantage to emerge. On the simple application measured here the tax is small, which is why C3 is competitive; the trade is expected to improve for C4 as applications grow more complex.

8.3 A procedure for adding C4 to an existing application

For a developer adding a view document to an application today (the hand-authored or spec-derived paths), the work reduces to a short, repeatable procedure:

1. **Enumerate the views.** List the distinct states the UI can be in (catalog, product, cart, checkout, confirmation, ...) — the same screens the human UI already has.
2. **For each view, expose three things:** the current *state* (the fields the view depends on), the relevant *entities* (the objects shown), and the available *actions*, each with its *enable-condition* and *argument constraints*. Crucially, reuse the human UI's existing disable and validation logic here — the predicate that greys out a button is the predicate that sets `enabled: false`, and the form validator that rejects an input is the argument schema.
3. **Serve the document** at an endpoint or resource — for instance as an MCP resource — so an agent can read the current view on each step.
4. **Keep the backend as the enforcing source of truth.** The document advertises validity; the backend still checks it. The surface is a projection, never the authority.
5. **Add a surface-vs-backend consistency check** (as in Section 4) so the document can never advertise an action the backend would reject. This is the test that makes the projection trustworthy and prevents the two surfaces from drifting apart over time.

8.4 Complexity summary

Table 4 summarizes the five paths on the three axes above and states when each is appropriate.

Table 4. Adoption paths for the view document (C4).

Adoption path	App effort	Coverage	Fidelity	When to use
Framework-emitted	Low at scale (one-time framework cost)	High	High (single declaration, no drift)	New applications; ecosystems adopting an agent-aware UI framework
Derived from a structured backend	Low–medium (write/maintain a projection)	Medium–high (bounded by spec coverage)	High where the spec is authoritative	Apps with GraphQL, form/validation schemas, or server-driven UI
Hand-authored per view	Medium per app	Whatever is authored	High (authored against backend state; test-pinned)	A few high-value apps or flows; the upper-bound reference
Compiled from the human surface	Near-zero (work is in the compiler)	Broad (any app with a human UI)	Lossy; must be validated against the backend	Existing/legacy apps that cannot be rebuilt
Model-extracted	Near-zero	Universal	Lowest; recurs at runtime; hallucination risk	Fallback when no other surface is reachable

Empirically validating the two *generated* paths — the compiled and model-extracted surfaces — against the hand-authored upper bound established in this study (how much coverage and fidelity they retain, and at what cost) is left to future work (Section 10).

9. Limitations

This is one model on one small, synthetic application with a ten-task set; statistical power is limited and the results are provisional. The C4 document is hand-authored (an upper bound, not an automatically generated surface). Image tokens are provider-dependent and here are folded into prompt tokens, so the screenshot (C1) cost is read from total input tokens. Latency is not yet measured. Refusal tasks can be passed by inaction and are reported separately. Temperature-zero decoding is not fully deterministic, which is why results are averaged over repeats.

10. Future work

Immediate: latency instrumentation and a timed re-run; a second and third general model for model-agnosticism; the constraint-pruning ablation; and an "ignored-affordance" metric. Longer term: a second, more complex application (the main external-validity lever); a validated Agent-Experience (AX) metric suite (the second paper); and methods for *automatically generating* the

agent surface (compiled from the human interface or model-extracted), measured against the hand-authored upper bound established here.

Appendix: reproducibility

The application, four-condition harness, deterministic grader, model registry, and the tooling used for every number above are in the repository; the exact command sequence is the reproducibility checklist in Section 5 ("Procedures"), and the environment is defined in `.cursor/`. See [docs/research-plan.md](#) for the full methodology and [docs/paper-outline.md](#) for section status.

References

Verified sources (to be formatted for the target venue):

- WebArena — <https://arxiv.org/abs/2307.13854>
- VisualWebArena — <https://aclanthology.org/2024.acl-long.50/>
- Mind2Web — <https://arxiv.org/abs/2306.06070>
- WebShop — https://proceedings.neurips.cc/paper_files/paper/2022/file/82ad13ec01f9fe44c01cb91814fd7b8c-Paper-Conference.pdf
- AndroidWorld — <https://arxiv.org/abs/2405.14573>
- MiniWoB++ — <https://arxiv.org/abs/1802.08802>
- Anthropic computer use — <https://www.anthropic.com/news/3-5-models-and-computer-use>
- UI-TARS — <https://arxiv.org/abs/2501.12326>
- SeeClick — <https://aclanthology.org/2024.acl-long.505/>
- UIFormer — <https://arxiv.org/abs/2512.13438>
- Prune4Web — <https://ojs.aaai.org/index.php/AAAI/article/download/40772/44733>
- A2UI — <https://a2ui.org/>
- Model Context Protocol — <https://modelcontextprotocol.io/>